Global Server Load Balancing and Regional Routing

send each user to the right region, and survive losing one

Watch the 4-min video

In one line: a global server load balancer (GSLB) picks the region for each user, by location, latency, health, load and data-residency rules, using DNS answers (simple, but failover waits for the TTL) or anycast (one IP everywhere, failover in seconds); the other regions need spare capacity and the data must be there too.

1 One region is too far and too fragile

Your app runs in Virginia. A user in Mumbai waits about 200 ms for every round trip, and a page that needs several round trips feels slow. A single region outage takes everyone offline. So you deploy to three regions, and a new question appears: how does each user reach the **right** region, and what happens when one goes dark?

2 The global load balancer

A **global server load balancer** (GSLB) decides which region or data centre serves each user. It sits **above** your regional load balancers. It weighs:

- where the user is, and the latency to each region;
- the health of each region (probes from many places);
- how loaded each region is;
- rules about where data may live (data residency).

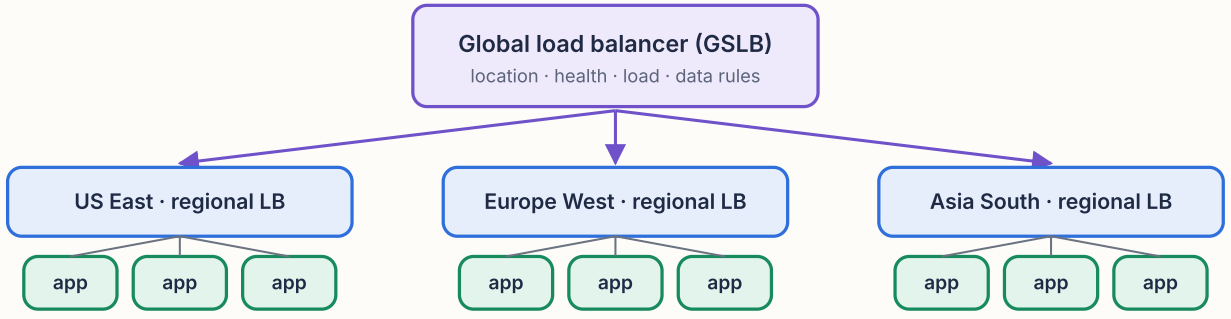


Figure 1. Two levels of load balancing: a global layer chooses the region, a regional load balancer chooses the server.

3 Two ways to steer: DNS and anycast

DNS-based: the user's resolver asks for your domain, and the GSLB answers with the IP of the chosen region. **Geo** routing picks by location, **latency** routing by the lowest measured latency, **weighted** routing splits traffic (Phase 2, lesson 17). AWS Route 53 and Azure Traffic Manager work this way. The catch: **caching**. Resolvers and clients keep the old answer until the TTL expires, so failover takes at least a TTL, often longer.

Anycast: every edge location announces the **same IP address**, and internet routing (BGP) carries each user to the nearest one. When a location withdraws its route, traffic shifts within seconds, with no DNS

caching involved. Cloudflare, Google's global load balancer and Azure Front Door use anycast at the edge, then forward to backend regions over their own network.

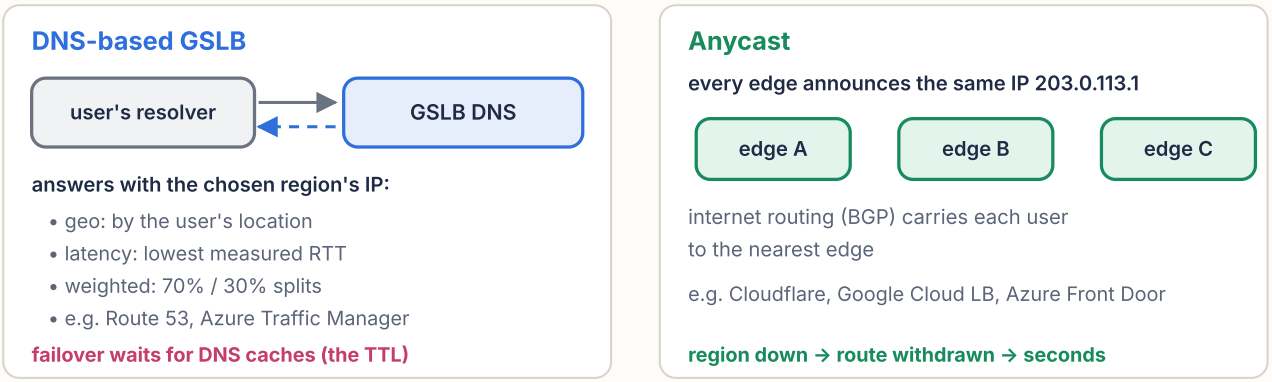


Figure 2. DNS steers with answers and is limited by caching; anycast steers with routing and fails over in seconds.

WORKED EXAMPLE: AZURE FRONT DOOR

Front Door has edge locations worldwide, all on one anycast address. A user in Mumbai connects to a nearby edge, which terminates TLS (saving round trips over the long distance) and forwards the request to the closest healthy backend region, Central India. Front Door probes each region every few seconds. If Central India fails its health probes, requests shift to Southeast Asia automatically, without any DNS change.

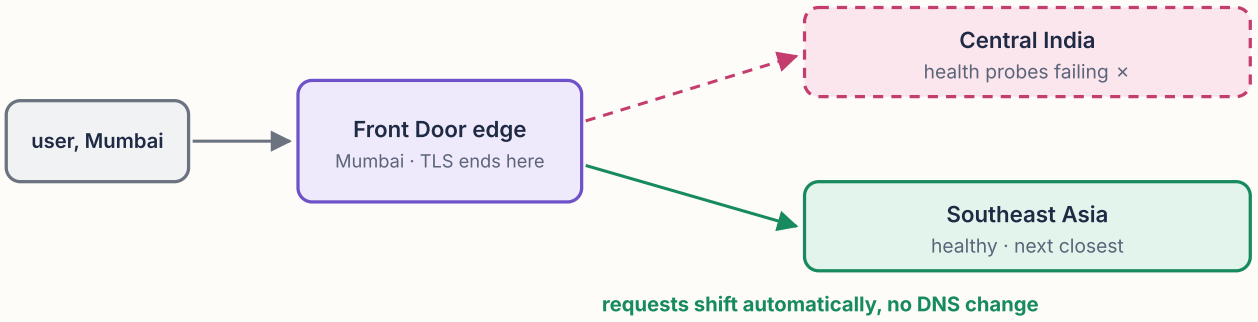


Figure 3. A regional failure behind an anycast edge: the edge stops sending to the failing region and uses the next closest one.

4 It's not only about speed: data

Data residency. Laws such as the EU's GDPR, or customer contracts, may require European users' data to stay in Europe. Those users must route to a European region, even if another one is faster at that moment.

Home regions. A user's data might live in one home region. Requests must go there (route by user, not just by location), or the data must be replicated where the user lands.

5 When a region fails

- **Spare capacity.** Traffic moves to the other regions, so they must be able to absorb it. With three regions, each needs headroom for half of a lost region's load.
- **Failover storms.** If everyone moves at once, the next region can be overloaded too, and fail. Shift gradually where possible, and shed low-priority load (lesson 10).

- **Writes.** If the failed region was the primary for some data, another region must be promoted. That's the next lesson.

COMMON MISTAKES

- Long DNS TTLs (an hour or a day) on records you expect to fail over.
- Routing purely by latency and breaking data-residency rules.
- Regions sized only for their own traffic, so a failover takes down the survivors.
- Health checks from a single location, so a network blip near the checker looks like a region outage.

6 Key terms

GSLB: global load balancer that picks a region

Latency routing: answer with the lowest-latency region

Anycast: one IP announced from many locations

Data residency: rules on where data may be stored

Geo routing: answer by the user's location

Weighted routing: split traffic by percentages

BGP: the routing protocol between internet networks

Failover storm: everyone moving at once and overloading the next region

7 Interview questions

Q1. How do you route users to the nearest region? BASIC

With a global load balancer: DNS-based geo or latency routing that answers with the best region's IP, or anycast IPs that send users to the nearest edge, which forwards to the closest healthy region.

Q2. What happens when a region fails? BASIC

Health checks mark it down and traffic shifts to other regions, within seconds with anycast and after the TTL with DNS, so those regions need spare capacity and the data must be available there.

Q3. Users in Germany must have their data stored in the EU. How does routing change? SCENARIO

Route them to an EU region by account or location rules, even if another region is faster, keep their data and backups in the EU, and fail over only to another EU region.

Q4. You use DNS failover with a 300 s TTL, but some users take 15 minutes to move. Why? SCENARIO

Some resolvers and clients cache longer than the TTL, and long-lived connections stay open. Use short TTLs, close connections on the failed side, or move to anycast at the edge.

Q5. Why might latency-based routing send a user to a far region? TRICKY

It measures from the user's DNS resolver, not the user; a public resolver far away (or without EDNS Client Subnet) makes the far region look closer. Anycast or client-measured latency avoids this.

Q6. Two of three regions must absorb a lost region. How much capacity does each need? TRICKY

Each normally carries a third of the load; after a failure, each carries half. So each region needs headroom for about 1.5× its normal load, plus margin for retries.

Next: SD-G-012 · Failover Strategies backendroadmap.com